[Dinu Barbu]

Context is **compiled**, not collected.

A context management system is not storage. It is a **compiler** that runs once per model call, and the thing it emits is thrown away immediately afterwards.

// an explainer

↳ dvinubius · dinubarbu.com

---

01 / the lesson

What a context management system actually is

A **context management system** is the runtime machinery that turns all potentially available information into the **smallest sufficient working set** for one particular model invocation.

Three words in that sentence do the work. **Smallest**, because tokens are budget. **Sufficient**, because the task still has to succeed. **One particular**, because the next invocation gets a different working set, built again from scratch.

fig. 1 · the six stages, and what happens after each run

01	02	03	04	05	06
authorize	retrieve	rank	transform	compress	assemble

After each run, persist and repeat.

The persist-and-repeat is what makes this a compiler rather than a queue — the run ends by leaving something behind for the run after it

---

02 / the reframe

Storage problem? No — an **optimization** problem.

The naive mental model treats the context window as a bucket: keep appending messages, files, and tool results until it overflows. The system described here replaces that with an explicit objective — and once there is an objective, the six familiar complaints about long sessions stop being symptoms and become the quantities being minimized.

fig. 2 · the same window, under two different mandates

before · accumulate	after · compile
Everything that ever entered the conversation stays visible. The window fills with: × irrelevant tokens × latency and cost × stale or contradictory information × information leakage × prompt-injection exposure × loss of important state when it finally overflows	Each invocation gets a freshly built working set, chosen to: ✓ **maximize** expected task performance ✓ **minimize** every one of them — simultaneously Those six failure modes aren't accidents of long conversations; they are the exact quantities the system is built to drive down, on every single call.

the dark panel is the state the doc argues for — not a better category of thing, a different one

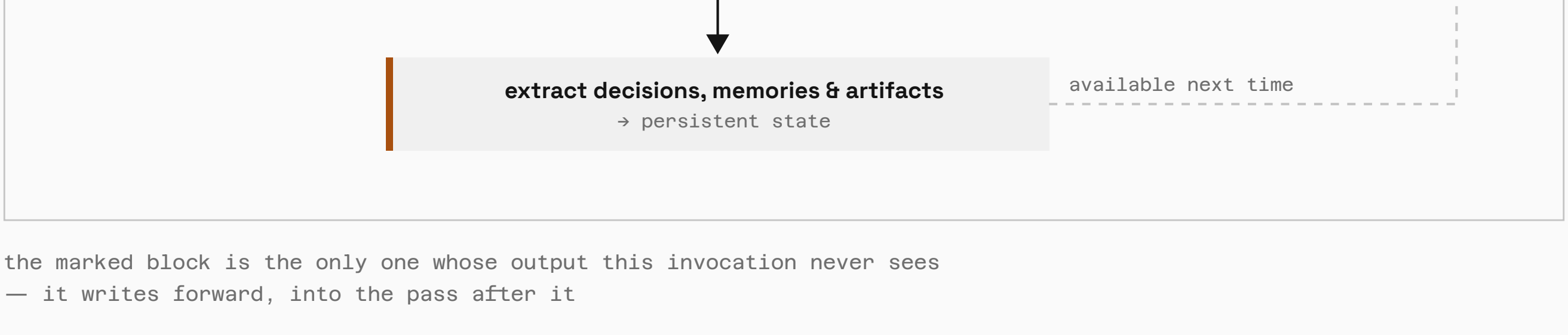
---

03 / the architecture

Six stages — and a loop back

The core architecture is a pipeline from **available information** to **model-visible context**. Crucially, it doesn't end at the model: what the invocation produces is extracted into persistent state, which becomes available information for the **next** compile pass. Every block below reads, and assembles what it read into this one call — except the last, which **writes**. That is the difference the marked rule points at, here and everywhere else in this piece.

fig. 3 · one compile pass, and the path back into the next one



the marked block is the only one whose output this invocation never sees — it writes forward, into the pass after it

fig. 4 · what each stage is responsible for

01 / authorize Determine what this user, agent, and task are allowed to access.	02 / retrieve Locate candidate sources, connectors, tools, memories, artifacts.	03 / rank Estimate which pieces are useful for the **present step**.
04 / transform Convert heterogeneous state into model-ready instructions, evidence, and messages.	05 / compress Trim, summarize, deduplicate, or defer to fit token, cost, and latency budgets.	06 / assemble Build the final structured input for the chosen model.

six peers — no stage is marked, because the order is the claim, not any one of them

> ### Why authorize comes first, not last
>
> Policy runs **before** retrieval. A system that retrieves first and filters afterwards has already loaded the forbidden data into a process that can leak it — and it has already paid for it. Put the same check at stage 01 and the data is never a candidate at all.

---

04 / the signature

context_t is a function, not a variable

The subscript **t** is the whole point — context is a **function evaluated at every step**, not a variable that grows. Eight inputs go into each evaluation:

fig. 5 · the compile signature

```
context_t = compile(
    current task,           // what step t is trying to achieve
    conversation state,     // recent + summarized history
    durable memory,         // preferences, decisions, learned facts
    available data,         // docs, code, records, tickets
    available tools,        // APIs, skills, delegable agents
    model characteristics,  // the chosen model shapes the format
    token budget,           // hard ceiling on cost & latency
    security policy         // what may be seen at all
)
```

the accented line is the signature itself — everything else is an argument to it

Note what's **parameters** here: the **model**, the **budget**, and the **security policy** are inputs to compilation — the same task compiles differently for a different model, a tighter budget, or a stricter policy. Change any one of them and you have not changed a setting; you have changed what gets built.

---

05 / what goes in

Seven classes in, three places to live

The system juggles seven distinct classes of context — each with different lifetime, authority, and risk:

fig. 6 · the seven classes, and the rule that governs all of them

Instructions system rules, org policies, project conventions	**Task state** objectives, plans, pending work, intermediate results	**Conversation state** recent messages, summarized history	**Memory** preferences, past decisions, learned facts
Evidence documents, code, records, emails, tickets, web results	**Capabilities** tools, APIs, skills, agents for delegation	**Artifacts** files, reports, patches produced during execution	**Each class earns its tokens per invocation** — none is "always in".

seven peers and one rule — the dark cell is not an eighth class

…and the distinction the system must never blur

fig. 7 · three words that are routinely used as one

Context What the model can see **now**. Ephemeral, rebuilt each step.	**Memory** Durable information that **may become** context later — if it wins ranking.	**Sources of truth** External systems facts can be **re-retrieved** from. Never confuse a cached copy with the source.

uncoloured on purpose — these are three categories, not three ranks

---

06 / one pass, traced

One compile pass, traced

An illustration of the pipeline on a familiar scenario: a coding agent's next step is **"fix the failing test."** Watch each stage earn its keep.

fig. 8 · six stages and a persist, on one concrete task

01 authorize	This task may read the repo and its tickets — not another team's incident channel. Policy runs **before** retrieval, so forbidden data is never even a candidate.
02 retrieve	Pull candidates: the failing test file, the module under test, the ticket, a memory noting a past decision about this module.
03 rank	The stack trace and the module outrank the ticket's boilerplate for **this** step. Ranking is per-step, not per-conversation.
04 transform	Heterogeneous state becomes model-ready form: code as annotated snippets, the ticket as a three-line summary, tools as structured definitions.
05 compress	Older conversation turns collapse into a summary; duplicate log lines are deduplicated; low-rank candidates are deferred, not deleted — the budget holds.
06 assemble	Instructions, task state, evidence, and tool definitions are linked into the structured messages this specific model expects.
…persist	**After the model runs:** "chose approach B over A" is extracted as a durable decision. Next invocation, it's **available information** — the loop closes.

the marked row is the same claim fig. 3 makes — the pass does not end at the model, and the six rows above it are read-only in comparison

Note what **deferred, not deleted** buys at stage 05. A low-rank candidate that is dropped is gone; one that is deferred stays in available information and can win ranking on the very next step, when the task has moved on and it is suddenly the useful thing.

---

07 / the compression

Four jobs in one system

The compact description: **"a query planner, memory manager, policy engine and linker for model inputs."** Nine responsibilities sort cleanly under those four hats:

fig. 9 · nine responsibilities under four hats

Query planner Discovery & ingestion · retrieval & ranking — find what could help and estimate what will.	**Policy engine** Identity & policy enforcement · provenance & conflict handling — who may see what, where facts came from, which version wins.
Memory manager Lifecycle management — what stays ephemeral, what becomes durable, what expires or must be revalidated.	**Linker** Context compilation · budget management · handoff semantics — build the invocation, fit the budget, and pass a sub-agent only the state it needs, never the whole parent conversation.
Cross-cutting: observability & evaluation. Record what context was selected and measure whether the selection improved or degraded the outcome — the pipeline is tunable only if it's observable.	

four hats and the one responsibility that sits across all of them

Keep the four terms straight

Context engineering — the discipline and design process.	**Context management system** — the runtime implementation.
Context compiler — the component that constructs one specific model invocation.	**Memory system** — the durable storage and recall subsystem.

---

08 / what to do with this

Six questions to ask of your own agent

Every question below is one stage of the pipeline turned back on the system you already run. If the honest answer is the one in the second line, that stage is missing — you have a bucket there, not a compiler.

01 **Does policy run before retrieval, or after?**
If you fetch first and filter the prompt afterwards, the forbidden data has already been loaded and already been paid for. **Authorize is stage 01.**

02 **Is anything in your prompt "always in"?**
A file pinned into every request is a class that stopped earning its tokens. **Each class earns its tokens per invocation.**

03 **Do you rank per step, or per conversation?**
Relevance decided once at the top of a session is stale by the third step. **Ranking is a function of the present step.**

04 **When you run out of room, do you drop or defer?**
Dropped candidates cannot come back when the task turns toward them. **Deferred is recoverable; deleted is not.**

05 **Where does a decision go after the model answers?**
If "chose B over A" survives only as a message in a transcript that will later be summarized away, the loop never closed. **Extract it into persistent state.**

06 **Can you replay which context a given call was given?**
Without that record you cannot tell an improvement from a regression, and every change to the pipeline is a guess. **Tunable only if observable.**

> ### Start with the two that pay immediately
>
> Log the selected context for one week — question 06 — and move your access check ahead of retrieval — question 01. The first tells you which of the other four are actually costing you; the second closes the one gap whose failure mode is a **leak** rather than a wasted token.

[Dinu Barbu] · Software Engineer — Agentic Developer Workflows
↳ dvinubius · dinubarbu.com